



DarknetSec: A novel self-attentive deep learning method for darknet traffic classification and application identification

Jinghong Lan^{a,b,*}, Xudong Liu^{a,b}, Bo Li^{a,b}, Yanan Li^c, Tongtong Geng^d

^a School of Computer Science and Engineering, Beihang University, Beijing, China

^b Beijing Advanced Innovation Center for Big Data and Brain Computing, Beihang University, Beijing, China

^c School of Computer and Information Security, Guilin University of Electronic Technology, Guilin, China

^d School of Economics and Management, Xidian University, Xi'an, China

ARTICLE INFO

Article history:

Received 20 October 2021

Revised 10 January 2022

Accepted 15 February 2022

Available online 17 February 2022

Keywords:

Darknet traffic

Convolutional neural network

Long short-term memory

Self-attention mechanism

Spatial-temporal features

Classification

ABSTRACT

Darknet traffic classification is crucial for identifying anonymous network applications and defending cyber crimes. Although notable research efforts have been dedicated to classifying darknet traffic by combining machine learning algorithms and elaborately designed features, current methods either heavily depend on hand-crafted features or overlook the global intrinsic relationships among the local features automatically extracted from different data positions, leading to limited classification performance. To tackle this issue, we propose DarknetSec, a novel self-attentive deep learning method for darknet traffic classification and application identification. Concretely, DarknetSec utilizes a cascaded model with a 1-dimensional Convolutional Neural Network (1D CNN) and a bidirectional Long Short-Term Memory (Bi-LSTM) network to capture local spatial-temporal features from the payload content of packets, while the self-attention mechanism is integrated into the abovementioned feature extraction network to mine the intrinsic relationships and hidden connections among the previously extracted content features. In addition, DarknetSec extracts side-channel features from payload statistics to enhance its classification performance. We evaluate DarknetSec on the CICDarknet2020 dataset, which is a representative of darknet traffic covering both Virtual Private Network (VPN) and The Onion Router (Tor) applications. Thorough experiments show that DarknetSec is superior to other state-of-the-art methods, achieving a multiclass accuracy of 92.22% and a macro-F1-score of 92.10%. Additionally, DarknetSec maintains its high accuracy when applied to other encrypted traffic classification tasks.

© 2022 Elsevier Ltd. All rights reserved.

1. Introduction

Darknet is described as an individual encrypted part of the Internet that can only be accessed with specific anonymity tools such as The Onion Router (Tor), Invisible Internet Project (I2P), Virtual Private Network (VPN), and JonDonym (generally known as Java Anon Proxy, JAP, or WebMix) (Montieri et al., 2018). The earliest case of using darknet can be traced back to 1971, when two students from the Massachusetts Institute of Technology and Stanford University used the Advanced Research Project Agency (ARPANET) to trade marijuana in the artificial intelligence laboratory of the Massachusetts Institute of Technology (Buxton and Bingham, 2015). Since the ARPANET was formed, the definition of

darknet has been extended to Peer-to-Peer (P2P) networks and private networks such as Tor (Barratt, 2015; Wood, 2009). After the publication of "The Darknet and the Future of Content Distribution" in 2002 (Biddle et al., 2002), the concept of darknet has gained wide acceptance in academic communities. It should be noted that in some previous literature, darknet is also defined as the unused address space of the Internet (also known as network telescopes, sinkholes, or blackholes) that has not been assigned to any hosts or devices (Fachkha and Debbabi, 2015; Iglesias and Zseby, 2017; Niranjana et al., 2020). However, such type of darknet is beyond the scope of this paper and will be left for future research.

Through encryption techniques and P2P connection networks, darknet provides anonymous services to individual users and can effectively combat routing eavesdropping and other traffic analysis techniques, thus ensuring the confidentiality and integrity of the communication data. In the first quarter of 2020, nearly two million worldwide users directly connected to Tor services, while approximately 50,000 users indirectly connected to Tor services

* Corresponding author at: School of Computer Science and Engineering, Beihang University, Beijing, China.

E-mail addresses: lanjh@act.buaa.edu.cn (J. Lan), liuxd@act.buaa.edu.cn (X. Liu), libo@act.buaa.edu.cn (B. Li), xngxliyanan@163.com (Y. Li), ggtt.jesse@veda.com (T. Geng).

through bridges (Habibi Lashkari et al., 2020). Due to its anonymous nature, darknet is widely used for criminogenic activities, such as drug dealing, arms smuggling, child pornography, terrorism and cyberattacks (e.g., botnets) (Adewopo et al., 2019; Al-Nabki et al., 2019). Therefore, accurately identifying darknet traffic and its corresponding application types is very meaningful for monitoring malware propagation early, detecting malicious activities and combating cybercrimes. Although private networks and encryption techniques provide individual users with anonymity, the interactive network traffic generated when an end user visits a darknet application can still reveal the intrinsic characteristics of the hidden network service (Dong et al., 2020; Lin et al., 2021; Shapira and Shavitt, 2021). Indeed, as an essential part of traffic engineering, network traffic classification can be used to identify the specific darknet application type that an end user communicates with.

Previous works on network traffic classification can be distributed into three main categories according to their analysis approaches: port-based, signature-based and statistical-based techniques (Xie et al., 2021). Port-based methods rely on the specific transport port numbers registered by the Internet Assigned Numbers Authority (IANA) to represent well-known services; such approaches are prone to failure due to the widespread use of dynamic ports and covert channel techniques. Signature-based methods generally use payload analysis techniques such as Deep Packet Inspection (DPI) to mine unique signatures from packet payloads and judge whether the traffic to be inspected contains a specific signature. However, due to the adoption of encrypted communication techniques, conventional signature-based methods fail to analyze darknet traffic. Statistical-based methods extract traffic features at the packet-level or flow-level and leverage machine learning algorithms to train a classifier to distinguish between different traffic types. Since machine learning has made great progress in many classification-related fields, statistical-based methods have become a research hotspot in network traffic classification, attracting widespread attention from both academia and industry (Pacheco et al., 2018).

At present, many researchers mainly adopt the technical route of executing data-driven methods to make full use of the powerful learning capabilities of machine learning algorithms to solve the darknet traffic classification problem. For instance, several conventional machine learning methods, including the Light Gradient Boosting Machine (LightGBM), K-nearest Neighbor (KNN) algorithm, Logistic Regression (LR), Random Forest (RF), Naive Bayesian (NB) model and Decision Tree (DTree) (Kumar et al., 2019; Montieri et al., 2018; 2019), have already been applied to this research field. However, in terms of model training, these methods either rely heavily on hand-crafted features or resort to a time-consuming feature selection process, thus leading to unstable performance when dealing with different network environments. As a further improvement, deep learning methods (Dong et al., 2020; Habibi Lashkari et al., 2020; Lin et al., 2021; Shapira and Shavitt, 2021; Singh et al., 2021) such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have also been introduced to automatically extract high-level features representations from network traffic, thus significantly reducing the reliance on domain experts and improving the generalizability for different network environments. Compared with conventional machine learning approaches, deep learning possesses an advantage in that features are not extracted manually but rather are automatically learned from the input data through a neural network. However, current deep learning-based methods mainly concentrate on extracting local spatial or temporal features from network traffic, whereas the global intrinsic dependency relationships and hidden connections among the local features extracted from different data positions are not fully considered, which ultimately leads to unsatisfied classification performance.

To tackle the aforementioned problems, in this paper, we propose DarknetSec, a novel self-attentive deep learning method for darknet traffic classification and application identification. Each part of DarknetSec processes the payload content or payload statistics of a network flow. Concretely, a self-attention-embedded 1D CNN and a bidirectional Long Short-Term Memory (Bi-LSTM) network are leveraged to extract local spatial-temporal features from the payload content of packets, and a multi-head self-attention module is designed to deal with the payload content in parallel. The output of the multi-head self-attention module and the local spatial-temporal features extracted by the self-attention-embedded 1D CNN and Bi-LSTM network are simultaneously fed into another attention module to automatically capture the global intrinsic dependency relationships and hidden connections among the local spatial-temporal features with different attention weights. Additionally, to enhance classification accuracy, a side-channel feature learning module is leveraged to extract feature representations from the payload statistics. Finally, the multiple abovementioned deep features are concatenated into a single vector and fed into a classification layer to obtain predictions. Thorough experiments on the CICDarknet2020 dataset show that the adoption of the self-attention mechanism, as well as the comprehensive consideration of both side-channel features and content features, significantly improves the accuracy of darknet traffic classification and application identification.

In summary, the main contributions of this paper are as follows:

- The 1D CNN, Bi-LSTM network and self-attention mechanism are integrated into a classifier to capture efficient local spatial-temporal features and mine the global intrinsic dependency relationships and hidden connections among them. To the best of our knowledge, this is the first study that addresses the darknet traffic classification problem using a multi-head self-attention module.
- We propose DarknetSec, a powerful end-to-end deep learning-based classification architecture using both side-channel features and content features. Our experiments show that DarknetSec can not only accurately classify darknet traffic but also maintain a high accuracy when applied to other encrypted traffic classification tasks.
- Extensive experiments conducted on the CICDarknet2020 dataset show that DarknetSec significantly outperforms other state-of-the-art methods in terms of the accuracy and F1-score metrics, demonstrating its superiority to effectively classify darknet traffic and identify its underlying application types.

The rest of this paper is organized as follows. Section 2 reviews the state-of-the-art related work. The system architecture of DarknetSec and its key components are detailed in Section 3. Experimental evaluations are presented in Section 4, and we conclude this paper in Section 5.

2. Related work

In this section, we review the related studies that investigate, analyze and classify darknet traffic, and we also shed light on encrypted traffic classification and spatial-temporal feature learning for classification-related tasks.

2.1. Darknet traffic analysis

In the last decade, academic communities have published a large number of representative works on darknet traffic analysis. For instance, Iliadis and Kaifas (2021) conducted a comparative study of feature selection and classification model chosen

for the accurate prediction of darknet traffic on the CICDarknet2020 dataset. [Shahbar and Zincir-Heywood \(2017\)](#) proposed to use a C4.5 classifier with network flow-level features to profile the users and applications on the I2P networks. [Rao et al. \(2018\)](#) proposed an unsupervised method with gravitational clustering to identify Tor network flows from non-anonymous network data. [Montieri et al. \(2018\)](#) used several conventional machine learning methods (NB, C4.5, and RF) to classify anonymous services (Tor, I2P, and JonDonym) and predict the specific application types from both flow-based and packet-based perspectives. More recently, deep learning having the ability of dealing with complex structures of high dimensional data has also been introduced to this research field. [Sarwar et al. \(2021\)](#) adopted modified Convolution-LSTM (CNN-LSTM) and Convolution-Gradient Recurrent Unit (CNN-GRU) deep learning techniques to conduct the darknet traffic classification and application identification task. [Sirinam et al. \(2019\)](#) proposed to use triplet networks with a KNN classifier to perform website fingerprinting against Tor anonymity systems. [Pour et al. \(2020\)](#) designed a binary classifier based on a CNN to identify compromised IoT devices by merely operating on darknet traffic, thus providing essential evidence for inferring ongoing orchestrated botnets. [Habibi Lashkari et al. \(2020\)](#) proposed DeeplImage, which converts the optimized statistical features selected by a forest of tree classifiers into gray images and trains a 2-dimensional CNN (2D CNN)-based classifier to classify darknet traffic. [Singh et al. \(2021\)](#) proposed a deep transfer learning architecture consisting of a pre-trained model and a baseline classifier to distinguish darknet traffic from benign traffic. [Lin et al. \(2021\)](#) proposed combining a 1D CNN and a stacked Bi-LSTM network to automatically extract high-level feature representations for the fine-grained classification of Tor traffic.

2.2. Encrypted traffic classification

Various works utilize machine learning-based methods to deal with encrypted traffic classification tasks, such as webpage fingerprinting ([Shen et al., 2020](#)), IoT device identification ([Pinheiro et al., 2019](#)), malicious communication detection ([Fang et al., 2021](#)) and encrypted application classification ([Dong et al., 2020](#); [Liu et al., 2020b](#); [Shapira and Shavitt, 2021](#); [Shen et al., 2021](#); [Yao et al., 2019](#)).

[Shen et al. \(2020\)](#) used the packet length information of client-server interactions to perform webpage fingerprinting. Specifically, three types of features, i.e., block features, sequence features, and statistical features, were extracted during the uplink-dominant stage and fed into a conventional machine learning model (e.g., RF, KNN or DTree) to generate a fine-grained classifier. [Pinheiro et al. \(2019\)](#) utilized packet length statistics and packet transmission rates to classify IoT and non-IoT devices. [Fang et al. \(2021\)](#) presented a communication channel-based method to detect malicious HTTPS traffic. Network packets with the same destination IP address and destination port were aggregated into a communication channel, and three types of features, i.e., distribution features, consistency features and statistical features, were extracted independently. Subsequently, a genetic algorithm-based method was used for feature selection, and a final classifier was built based on the RF algorithm.

For the encrypted application classification, recent studies have tended to utilize the powerful non-linear learning abilities of deep learning algorithms to obtain effective feature representations automatically, including 2D CNN ([Shapira and Shavitt, 2021](#)), 1D CNN with a bidirectional Gate Recurrent Unit (Bi-GRU) network ([Dong et al., 2020](#)), LSTM network with a hierarchical attention mechanism ([Yao et al., 2019](#)), graph neural network (GNN) ([Shen et al., 2021](#)), and Bi-GRU network with an attention mechanism ([Liu et al., 2020b](#)). It is worth noting that several exist-

ing methods have already adopted the attention mechanism to pay increased attention to the features that contribute most to traffic classification. However, instead of fully mining the global internal dependencies and hidden connections among the extracted local features during the feature extraction process, these methods only apply a single attention layer to the output feature vector, thus leaving much room for improvement ([Dong et al., 2020](#); [Liu et al., 2020b](#); [Yao et al., 2019](#)). In addition, to the best of our knowledge, the multi-head self-attention mechanism has not been used for darknet traffic classification in previous studies.

2.3. Spatial-temporal feature learning

Spatial-temporal feature learning is widely used in classification-related tasks, such as image classification, fault diagnosis, traffic flow prediction and network intrusion detection. Here, we only discuss those works that are closely related to this paper. [Hassan et al. \(2020\)](#), [Kanna and Santhi \(2021\)](#), [Xie et al. \(2020\)](#) used a CNN and a LSTM network successively to extract the spatial-temporal features of network traffic for intrusion detection. Specifically, [Hassan et al. \(2020\)](#) proposed a hybrid deep learning model to detect network intrusions based on a CNN and a weight-dropped LSTM (WDLSTM) network. The deep CNN was used to extract spatial features from network traffic data, and the WDLSTM was used to extract long-term temporal dependencies from the features extracted by the CNN. [Kanna and Santhi \(2021\)](#) proposed a unified model of an optimized CNN (OCNN) and a hierarchical multi-scale LSTM (HMLSTM) network to extract spatial-temporal features from the input network flows, while the Lion Swarm Optimization (LSO) algorithm is adopted to fine-tune the hyperparameters of the former OCNN. [Xie et al. \(2020\)](#) proposed HSTF-Model, an HTTP-based Trojan detection model based on the hierarchical spatial-temporal features of traffic at both packet-level and network flow-level. The authors used a standard CNN to extract spatial information and an LSTM network to extract temporal information. In terms of fault diagnosis, some representative works like ([Yang et al., 2020a](#); [2020b](#)) leveraged a cascaded model of a CNN and a GRU network to capture the spatial relations and long-term temporal dependencies among the time series signals collected from multiple sensors. Similarly, researchers also used the combination of CNN and RNN for traffic flow prediction in the intelligent transportation systems ([Liu et al., 2020a](#); [Zheng et al., 2020](#)).

Compared with conventional machine learning methods, deep learning can capture nonlinear features from the input data more effectively. Additionally, in terms of classification tasks, the cascade of CNN and RNN has also been proven to be suitable for extracting discriminative and expressive feature representations from the original input data with spatial-temporal characteristics (e.g., network traffic, time series of sensor signals). Furthermore, [Sarwar et al. \(2021\)](#) compared the performance of conventional machine learning and deep learning methods especially for darknet traffic classification, and demonstrated that deep learning is more accurate than conventional machine learning methods. Therefore, this paper takes a 1D CNN and a Bi-LSTM network as the basic unit to learn local spatial-temporal feature representations from the input network flows, and the self-attention mechanism is integrated into the deep feature extraction process to fully mine the inherent dependencies of the extracted local spatial-temporal features, thus contributing to capturing global and discriminative feature representations for fine-grained darknet traffic classification. Besides, instead of using a CNN with a fixed convolution kernel size, we use multiple convolution kernels of different sizes to extract different local spatial features of the input network flows and combine them to realize the accurate darknet traffic identification.

Table 1
List of notations.

Notation	Description
P_i	The i th packet of a network flow with only payload content
b_j^i	The j th byte of the i th packet
N	Number of packets kept in a network flow for content feature extraction
M	Number of bytes kept in a packet for content feature extraction
$\mathbf{x}_{sta}$	Statistical features
$\mathbf{x}_{seq}$	Sequential features
$\mathbf{x}_{scf}$	Side-channel features
$\mathbf{x}_{content}$	Content features
L	Size of the input packet length sequence
H	Number of heads in the multi-head self-attention module
$\mathbf{K}$	Key matrix of the self-attention mechanism
$\mathbf{Q}$	Query matrix of the self-attention mechanism
$\mathbf{V}$	Value matrix of the self-attention mechanism
$\mathbf{o}_{scf}$	Output of the side-channel feature learning module
$\mathbf{o}_{msa}$	Output of the multi-head self-attention module
$\mathbf{o}_{lstf}$	Output of the local spatial-temporal feature learning module
$\mathbf{o}_{acff}$	Output of the attentive content feature fusion module
$\hat{\mathbf{y}}$	Predicted probability vector of being a specific application type
$\mathbf{y}$	True label of ground truth presented in a one-hot encoding vector

3. The proposed DarknetSec

In this work, we propose DarknetSec, a self-attentive deep feature learning architecture, to classify darknet traffic and identify its specific application types. We present the overview and design details of each layer in DarknetSec. In summary, DarknetSec is a typical end-to-end classification solution for darknet traffic that outputs the application type of each network flow. The notations introduced in this paper are presented in Table 1.

3.1. DarknetSec overview

As shown in Fig. 1, DarknetSec is composed of three basic layers: a preprocessing layer, a feature learning layer and a classification layer. The preprocessing layer takes raw packets as inputs and splits them into network flows. Then, two types of features, i.e., content and side-channel features, are simultaneously extracted and converted into standard normalized vectors, which are fed into the feature learning layer. The feature learning layer consists of a side-channel feature learning module, a multi-head self-attention module, a local spatial-temporal feature learning module, and an attentive content feature fusion module. For the side-channel feature learning module, the side-channel features (statistical and sequential features) are fed into a Multilayer Perceptron (MLP) network to obtain abstract and high-level representations from the inputs, which will be proven to be helpful for improving the classification performance in Section 4.3. The content features are duplicated into two copies, which are fed into the multi-head self-attention module and the local spatial-temporal feature learning module to generate embedded feature representations. After that, the attentive content feature fusion module is utilized to fuse the outputs of the two content feature learning modules to highlight the intrinsic relationships among the learned spatial-temporal features. In DarknetSec, the abovementioned feature learning modules focus on different modalities of network traffic. The side-channel feature learning module summarizes the statistical and sequential characteristics from a global perspective, whereas the multi-head self-attention module and local spatial-temporal feature learning module present fine-grained content representations for darknet traffic classification. The classification layer concatenates the outputs of the side-channel feature learning module and the attentive content feature fusion module. It is composed of a dense layer and a softmax layer. The focal loss between the true labels and prediction probabilities is utilized for the overall training process.

3.2. Preprocessing layer

In this subsection, we focus on discussing the selection of suitable features to handle the darknet traffic classification problem. We select two types of features from the input network traffic, i.e., content features and side-channel features. Since raw network traffic is usually stored in pcapng or pcap files, whose formats cannot be directly used for machine learning, preprocessing is essential to split the raw packets into network flows. In this paper, a network flow is defined as a set of packets belonging to the same transport-level communication that have the same five-tuples (source IP address, destination IP address, source port, destination port, and transport-layer protocol) in both directions, i.e., the source IP address/port can be exchanged with the destination IP address/port correspondingly. This definition of a network flow is widely used in network traffic classification-related literature.

Although the protocol fields of the network layer and transport layer are essential parts of a packet, they are mainly designed for network transmission rather than application identification. Furthermore, the protocol fields below the application layer contain little effective information and cannot adequately provide discriminative features for fine-grained traffic classification. Consequently, when extracting five-tuple network flows, we remove the packet header and retain only the application layer data for each packet. Moreover, packets with no application layer data, malformed packets, loop packets, and retransmitted packets are discarded. As an example, the statistical distribution of network flows containing application layer data in the CICDarknet2020 dataset is shown in Fig. 2. Despite of the imbalance of traffic categories, we still can obtain enough training samples to build an accurate traffic classifier. Note that the same packet preprocessing approach was also used by Dong et al. (2020), in which the VPN part of the CICDarknet2020 dataset was used to perform application classification to demonstrate that only using the application layer data of network packets as inputs is efficient for the encrypted traffic classification task.

Content features. Content features are extracted from the payload content of each selected packet. Since the number of packets contained in a network flow and the size of each packet are commonly uncertain and are determined by the specific application type under study, a typical method is to select the first N packets of a network flow while retaining the first M bytes for each packet. N and M are the two hyperparameters of DarknetSec. Note that if a network flow contains more than N packets, truncation is per-

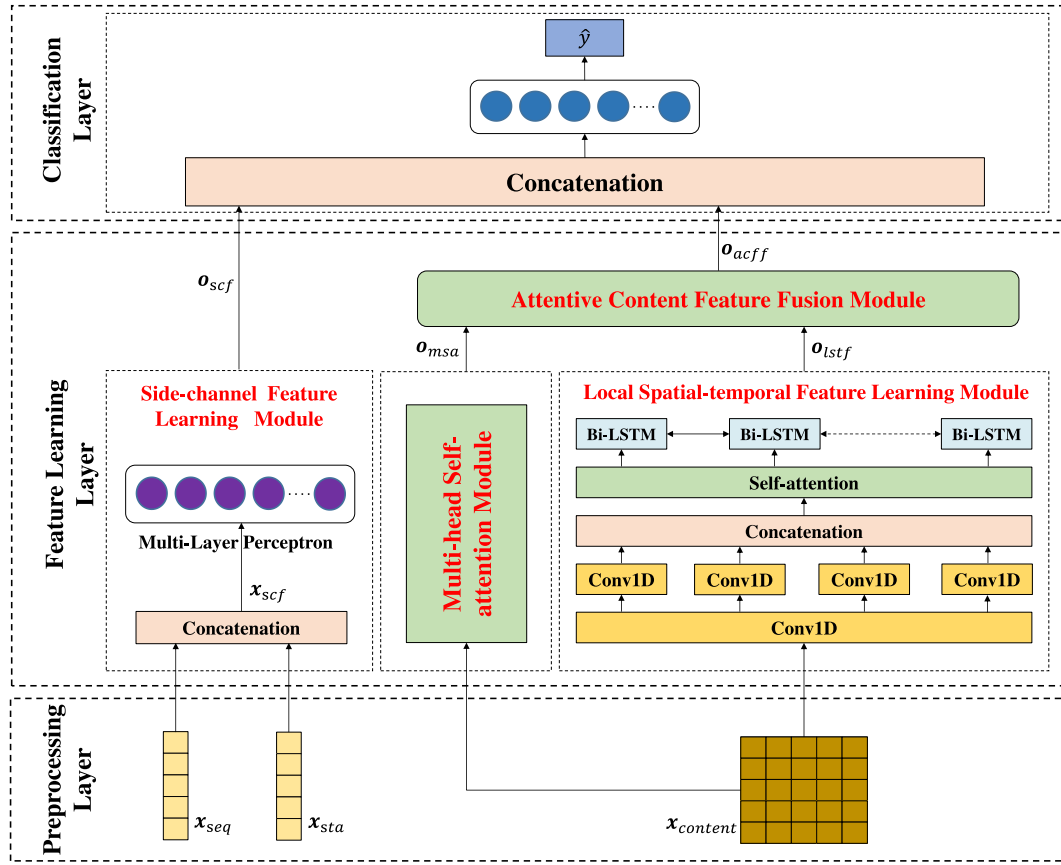


Fig. 1. The architecture of DarkNetSec.

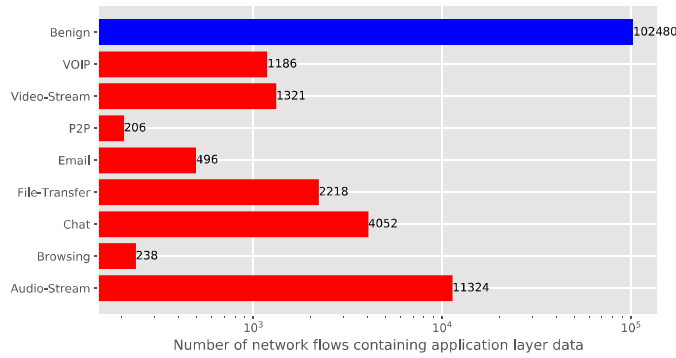


Fig. 2. Distribution of network flows containing application layer data in the CIC-Darknet2020 dataset.

formed; otherwise, it is padded with zeros. Similarly, if a packet contains more or fewer than M bytes, preprocessing will be performed in the same way.

As shown in Fig. 3, in terms of content feature extraction and vectorization, the encrypted input packet sequence is converted into the flow packet format to adapt to the local spatial-temporal feature mining from packet bytes. The payload content of a network flow is formalized as

$$\mathbf{x}_{content} = [\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_N], N \in \mathbb{Z}_+ \quad (1)$$

$$\mathbf{P}_i = [b_1^i, b_2^i, \dots, b_M^i], i \in [1, N] \quad (2)$$

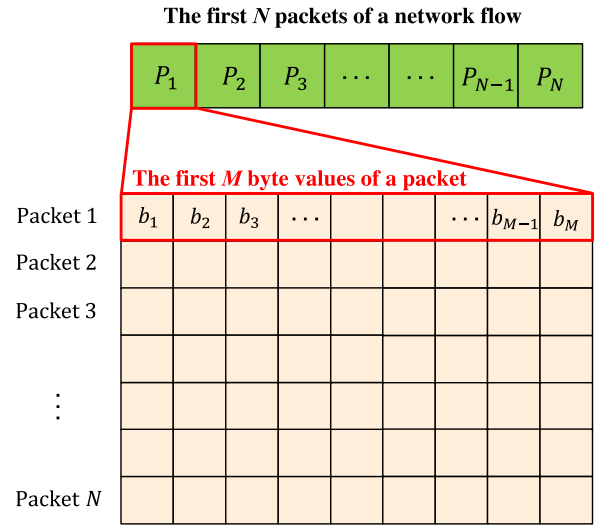


Fig. 3. The payload vectorization method for a network flow.

where $\mathbf{x}_{content}$ represents the content features of a network flow, $\mathbf{P}_i$ is the i th packet with only application layer data, and $b_j^i \in [0, 0xff]$, $i \in [1, N]$, $j \in [1, M]$ denotes the j th byte of the i th packet.

Then, data normalization is performed to map the numeric packet bytes to $[0,1]$ by dividing all byte values by $0xff$.

Side-channel features. Side-channel features are defined as the information that can be extracted from encrypted network traffic without decryption (Stergiopoulos et al., 2018); they are composed of statistical and sequential features. Multiple statistical fea-

Table 2
Detailed descriptions of the selected side-channel features.

Feature Type	Feature description	Number
Statistical features	Flow duration	35
	Time intervals among packets(max, min, mean, standard deviation, median)	
	Packet length statistics(max, min, mean, standard deviation, median)	
	Inbound packet length statistics(max, min, mean, standard deviation, median)	
	Outbound packet length statistics(max, min, mean, standard deviation, median)	
	Number of packets/bytes	
	Number of inbound packets/bytes	
	Number of outbound packets/bytes	
	Number of packets/bytes per second	
	Number of inbound packets/bytes per second	
	Number of outbound packets/bytes per second	
Sequential features	Ratio of inbound and outbound packets/bytes	L
	Packet length vector for the first L packets	

tures are extracted from the payload content of packets based on the granularity of a network flow, and the packet length vector for the first L packets of a network flow is selected as the sequential features. Note that L is a hyperparameter and will be discussed in Section 4.2. Detailed descriptions of the selected side-channel features is presented in Table 2. Additionally, to eliminate the negative effects of different side-channel feature value ranges on the classifier, as defined in Eq. (4), a normalization method called the “min-max” is utilized for the side-channel data preprocessing. Finally, the statistical features, sequential features and side-channel features are denoted as $\mathbf{x}_{sta}$, $\mathbf{x}_{seq}$, and $\mathbf{x}_{scf}$, respectively.

$$\bar{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3)$$

where $x_{\min}$ and $x_{\max}$ denote the minimum and maximum values of feature x , respectively.

3.3. Feature learning layer

To obtain high-level representations from the original side-channel features and content features, we design a side-channel feature learning module, a multi-head self-attention module, a local spatial-temporal feature learning module, and an attentive content feature fusion module.

First, the statistical features $\mathbf{x}_{sta}$ and sequential features $\mathbf{x}_{seq}$ are concatenated into a single vector as the side-channel features $\mathbf{x}_{scf}$, which are subsequently fed into an MLP network. The content features $\mathbf{x}_{content}$ are duplicated into two copies and simultaneously handled by the multi-head self-attention module and local spatial-temporal feature learning module. Then, the outputs of the two abovementioned content feature learning modules flow to an attentive content feature fusion module to capture the corresponding weighted spatial-temporal representations. Finally, the side-channel feature learning module generates an output $\mathbf{o}_{scf}$, while the attentive content feature fusion module generates an output $\mathbf{o}_{acff}$. These two output vectors are fed into the classification layer to generate the final prediction.

3.3.1. Side-channel feature learning module

The MLP is one of the most basic type of neural networks. The neurons in two adjacent layers are fully connected, so the MLP is also called the fully connected neural network. An MLP classifier is a typical machine learning model with strong nonlinear representation ability that is widely used in classification-related tasks. For the side-channel features $\mathbf{x}_{scf}$, we leverage an MLP network to mine high-level representations from the input data. $\mathbf{o}_{scf}$ is the output of this module, as shown in Fig. 1($\mathbf{o}_{scf} = \mathbf{h}_Z$). More specifically, the MLP used for side-channel feature learning can be formalized as

malized as

$$\mathbf{h}^i = \begin{cases} \sigma(\mathbf{W}^i \mathbf{x}_{scf} + \mathbf{b}^i), & i = 1 \\ \sigma(\mathbf{W}^i \mathbf{h}_{i-1} + \mathbf{b}^i), & 1 < i \leq Z \end{cases} \quad (4)$$

where $\mathbf{W}^i$, $\mathbf{b}^i$, and $\mathbf{h}^i$ denote the weight matrix, bias vector, and output vector of layer i , respectively. $\sigma(\cdot)$ is a non-linear activation function. The MLP is introduced to capture an abstract representation from the original numerical side-channel features. This representation is combined with the output of the attentive content feature fusion module to determine the final classification.

3.3.2. Multi-head self-attention module

For traffic classification tasks, network traffic is often sequential, and internal dependencies commonly exist among the payload content of packets. Thus, we need to develop an appropriate approach that not only pays attention to extracting spatial-temporal representations from the payload content, but also considers their inherent dependencies. Sequence encoding based on CNNs or RNNs is essentially a type of local encoding method that only models the local dependencies among the input data. Although RNNs can theoretically establish long-distance dependencies, they are limited by the information storage capacity and vanishing gradient problems, thus resulting in only short-distance dependencies. In addition, improved RNNs such as LSTM also have some shortcomings. For example, LSTM considers the later features to be more important than the previous ones. In this paper, inspired by the network structure of Transformer (Vaswani et al., 2017), which is a sequence learning model based on attention mechanisms, we leverage a multi-head self-attention network to mine the multi-dimensional dependencies inherent within a network flow and dynamically generate attentive weights for different data positions of the input network flows. The multi-head self-attention network can adaptively pay more attention to specific parts of the input that are related to the classification and give less attention to the irrelevant parts, thus contributing to better capturing complex spatial-temporal representations from the input network flows.

The self-attention mechanism usually adopts a query-key-value (QKV) mode, whose calculation procedure is shown in Fig. 4. We first map the input $\mathbf{x}_{content} \in \mathbb{R}^{N \times M}$ linearly into three different spaces to obtain a query matrix $\mathbf{Q}$, a key matrix $\mathbf{K}$, and a value matrix $\mathbf{V}$. This linear mapping can be formalized as

$$\mathbf{Q} = \mathbf{W}_q \mathbf{x}_{content} \in \mathbb{R}^{D_q \times M} \quad (5)$$

$$\mathbf{K} = \mathbf{W}_k \mathbf{x}_{content} \in \mathbb{R}^{D_k \times M} \quad (6)$$

$$\mathbf{V} = \mathbf{W}_v \mathbf{x}_{content} \in \mathbb{R}^{D_v \times M} \quad (7)$$

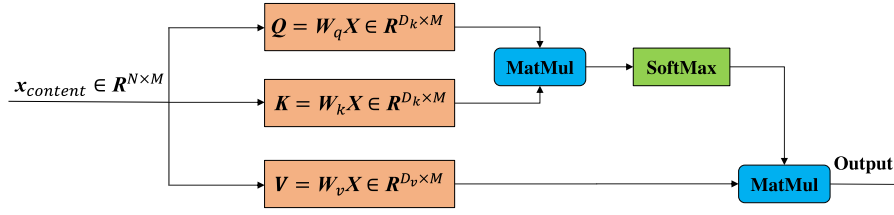


Fig. 4. The calculation procedure of the self-attention mechanism model.

where $\mathbf{W}_q \in \mathbb{R}^{D_k \times N}$, $\mathbf{W}_k \in \mathbb{R}^{D_k \times N}$, and $\mathbf{W}_v \in \mathbb{R}^{D_v \times N}$ denote the three parameter matrices of the linear mapping, which are randomly initialized and updated through backpropagation during the training process.

The query matrix $\mathbf{Q}$ and key matrix $\mathbf{K}$ are used to generate a distribution of attention weights, while the value matrix $\mathbf{V}$ is used to obtain the selected information. The output matrix is calculated as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{V} \cdot \text{Softmax}\left(\frac{\mathbf{K}^T \mathbf{Q}}{\sqrt{D_k}}\right) \quad (8)$$

$$\text{Softmax}(\mathbf{X}_{ij}) = \frac{\exp(\mathbf{X}_{ij})}{\sum_k \exp(\mathbf{X}_{kj})} \quad (9)$$

where $\text{Softmax}()$ represents a column-wise-normalized function.

In this paper, we employ a multi-head self-attention module to obtain better nonlinear representations; this module uses multiple query vectors to select multiple sets of information from the input data in parallel, and each attentive head focuses on a different part of the input data. This provides an approach to fully mine the information from multiple representation subspaces at different positions. Specifically, the multi-head self-attention module can be formalized as

$$\mathbf{o}_{msa} = \text{Multihead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_H) \mathbf{W}_{msa} \quad (10)$$

$$\text{head}_i = \text{Attention}(\mathbf{W}_{q_i} \mathbf{x}_{content}, \mathbf{W}_{k_i} \mathbf{x}_{content}, \mathbf{W}_{v_i} \mathbf{x}_{content}) \quad (11)$$

where H and $\mathbf{o}_{msa} \in \mathbb{R}^{D_v \times M}$ denote the head number and output of the multi-head self-attention module, respectively, and $\mathbf{W}_{msa} \in \mathbb{R}^{H \times M}$, $\mathbf{W}_{q_i} \in \mathbb{R}^{D_k \times N}$, $\mathbf{W}_{k_i} \in \mathbb{R}^{D_k \times N}$, and $\mathbf{W}_{v_i} \in \mathbb{R}^{D_v \times N}$ are parameter matrices.

3.3.3. Local spatial-temporal feature learning module

As shown in Fig. 1, the local spatial-temporal feature learning module consists of a two-layer Conv1D network, a self-attention mechanism layer, and a Bi-LSTM network. It is a core component of DarknetSec that aims to capture the local spatial-temporal patterns of an input network flow. The detailed descriptions are presented as follows.

Conv1D. We utilize Conv1D to capture the spatial dependencies of $\mathbf{x}_{content}$. Conv1D is composed of a 1D CNN layer, a batch normalization layer, and a Rectified Linear Unit (ReLU) activation function, which can be formalized as

$$\mathbf{o}_{conv1d}(\mathbf{x}) = g_{relu}(g_{bn}(g_{conv}(\mathbf{x}))) \quad (12)$$

$$g_{conv}(\mathbf{x}) = \mathbf{W}_{conv} \otimes \mathbf{x} + \mathbf{b}_{conv} \quad (13)$$

where $\mathbf{x}$ and $\mathbf{o}_{conv1d}(\mathbf{x})$ denote the input and output of a Conv1D block, respectively. $g_{conv}()$, $g_{bn}()$, and $g_{relu}()$ are the 1-dimensional convolution function, batch normalization function, and ReLU activation function, respectively. $\mathbf{W}_{conv}$, $\mathbf{b}_{conv}$, and $\otimes$ denote the parameter matrix, bias vector, and convolution operation, respectively. In

our model, $g_{conv}()$ is leveraged to capture local spatial representations from the input content features. It is worth noting that since the spatial feature dimension is not large, a pooling layer is not used after the 1-dimensional convolution function.

Batch normalization is a technique used to improve the performance and stability of neural networks. It generates an input with zero mean/unit variance for any layer of a specific neural network. Even if the mean and variance change during the training process, batch normalization can standardize the data adaptively, thus facilitating gradient propagation and speeding up the training process. The calculation process of normalization can be formalized as

$$g_{bn}(x_1, x_2, \dots, x_{N_{batch}}) = (\alpha \hat{x}_1 + \beta, \alpha \hat{x}_2 + \beta, \dots, \alpha \hat{x}_{N_{batch}} + \beta) \quad (14)$$

$$\hat{x}_i = \frac{x_i - \mu_b}{\delta_b + \xi} \quad (15)$$

$$\mu_b = \frac{1}{N_{batch}} \sum_{i=1}^{N_{batch}} x_i \quad (16)$$

$$\delta_b = \sqrt{\frac{1}{N_{batch}} \sum_{i=1}^{N_{batch}} (x_i - \mu_b)^2} \quad (17)$$

where $\mathbf{x}_{bn} = x_1, x_2, \dots, x_{N_{batch}}$ denotes the input of the batch normalization function, and μ_b and δ_b are the mean and standard deviation of $\mathbf{x}_{bn}$, respectively. $\alpha \in \mathbb{R}^+$ and $\beta \in \mathbb{R}$ are trainable neural network parameters. ξ is an arbitrarily small constant.

As shown in Fig. 1, after the two-layer Conv1D network, a self-attention layer is placed before the Bi-LSTM network to associate the positions of the local spatial features obtained by the Conv1D network, which enriches the input features for the following Bi-LSTM network.

Bi-LSTM. After the two-layer Conv1D network and self-attention layer, we apply a Bi-LSTM network to perform comprehensive and thorough feature extraction for $\mathbf{x}_{content}$ in view of the temporal dependencies within a network flow. Bi-LSTM consists of a forward LSTM and a backward LSTM. Specifically, the calculation process of an LSTM unit can be formulated as

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (18)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (19)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (20)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (21)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (22)$$

where $\mathbf{W}_i, \mathbf{U}_i, \mathbf{W}_f, \mathbf{U}_f, \mathbf{W}_o, \mathbf{U}_o, \mathbf{b}_i, \mathbf{b}_f, \mathbf{b}_o$ denote trainable network parameters, $\mathbf{x}_t$ is the input vector at time step t , $\sigma()$ is the

element-wise sigmoid activation function and $\tanh()$ is the hyperbolic tangent activation function. $\odot$ denotes element-wise multiplication.

A forward hidden state $\vec{h}_t$ and backward hidden state $\overleftarrow{h}_t$ are generated at time step t . The final hidden state at time step t is obtained by concatenating the forward and backward hidden states, i.e., $h_t = [\vec{h}_t; \overleftarrow{h}_t]$. The hidden state h_t contains high-level spatial-temporal feature representations obtained from raw content features. In our model, we take all the hidden states of the Bi-LSTM network as the output of the local spatial-temporal feature learning module, which is denoted as o_{lstf} .

3.3.4. Attentive content feature fusion module

Composed of an approximate self-attention layer and a dense layer, the attentive content feature fusion module is designed to fuse the outputs of the multi-head self-attention module and the local spatial-temporal feature learning module, thereby helping to obtain the inner dependency relationships among the different data positions of a network flow from a global perspective and contributing to the learning of comprehensive content features. More specifically, the output of the multi-head self-attention module serves as the key matrix K and query matrix Q , while the output of the local spatial-temporal feature learning module serves as the value matrix V . The attentive content feature fusion module can be further formalized as

$$\begin{aligned} o_{acff} &= \text{Attentive}(o_{msa}, o_{lstf}) \\ &= W_{acff}(o_{lstf} \cdot \text{Softmax}(\frac{o_{msa}^T o_{msa}}{\sqrt{d_{msa}}})) + b_{acff} \end{aligned} \quad (23)$$

where d_{msa} is the dimensionality of o_{msa} used to control the scale, and o_{msa} , o_{lstf} , and o_{acff} denote the outputs of the multi-head self-attention module, local spatial-temporal feature learning module and attentive content feature fusion module, respectively. W_{acff} and b_{acff} respectively denote the parameter matrix and bias vector of the dense layer used to compress o_{acff} to a specified dimensionality.

3.4. Classification layer

Composed of a dense layer and a softmax layer, the classification layer combines the outputs of the side-channel feature learning module and the attentive content feature fusion module. Considering the class imbalance problem in darknet traffic classification, we take the focal loss between the one-hot encoding vectors of the true labels and the predicted probability distribution vectors as the loss values of the backpropagation during the deep learning-based training process. The core idea of the focal loss function is that samples which are difficult to classify are important for the entire classification model.

$$o_c = W_c \cdot \text{Concat}(o_{scf}, o_{acff}) + b_c \quad (24)$$

$$\hat{y} = \text{softmax}(o_c) \quad (25)$$

where o_{scf} is the output vector of the side-channel feature learning module, and o_{acff} is the output vector of the attentive content feature fusion module. W_c and b_c are the weight matrix and bias vector of the dense layer, respectively. o_c and $\hat{y}$ denote the outputs of the dense layer and the predicted probability vector, respectively. The focal loss function for a C -class classification task can be formalized as

$$\text{FL}(\hat{y}, y, \alpha, \gamma) = -\alpha(1 - \hat{y}_c)^\gamma \log(\hat{y}_c) \quad (26)$$

where $y \in \mathbf{R}^C$ and $\hat{y} \in \mathbf{R}^C$ denote the one-hot vector of the true label and the predicted probability vector, respectively. α is the class

weight, γ is the focusing parameter, and c denotes the label of the ground truth. Note that $y_c = 1$ and that other $y_i (i \neq c)$ values are all equal to 0 in $y \in \mathbf{R}^C$.

By combining the side-channel feature learning and spatial-temporal content feature learning components, DarknetSec provides a final prediction for each network flow. Overall, the detailed workflow of DarknetSec is presented in Algorithm 1.

Algorithm 1: The workflow of DarknetSec.

Input: 5-tuple network flows extracted from raw network traffic

Output: The predicted type of each network flow

- 1 Generating $x_{content}$, x_{sta} and x_{seq} ;
- 2 Initializing neural network parameters;
- 3 **for** each training iteration **do**
- 4 Selecting a batch of training samples;
- 5 $x_{scf} = \text{Concat}(x_{sta}, x_{seq})$;
- 6 $o_{scf} = \text{MLP}(x_{scf})$;
- 7 $o_{msa} = \text{Multi_head_self_attention}(x_{content})$;
- 8 $o_{lstf} = \text{Bi_LSTM}(\text{self_attention}(\text{Two_layer_Conv1D}(x_{content})))$;
- 9 $o_{acff} = \text{Attentive}(o_{msa}, o_{lstf}) = W_{acff}(o_{lstf} \cdot \text{Softmax}(\frac{o_{msa}^T o_{msa}}{\sqrt{d_{msa}}})) + b_{acff}$;
- 10 $o_c = \text{Dense}(\text{Concat}(o_{scf}, o_{acff}))$;
- 11 Obtaining $\hat{y}$ using a softmax layer: $\hat{y} = \text{softmax}(o_c)$;
- 12 Updating the trainable parameters of DarknetSec for the current batch of training samples with the focal loss;
- 13 **for** each test iteration **do**
- 14 Selecting a batch of test data;
- 15 Calculating and saving $\hat{y} = \text{argmax}(\hat{y})$ using the trained parameters;
- 16 **return** $\hat{y}$ for all test network flows;

4. Experimental evaluation

In this section, we are dedicated to evaluate the performance of DarknetSec. We conduct most experiments on the CDarknet2020 dataset, which is composed of VPN and Tor applications and acts as a real representative of darknet traffic by merging two public datasets, namely, ISCTXor2016 and IS-CXVPN2016 (Habibi Lashkari et al., 2020). Network flow samples are divided into two types: benign and darknet types, and the numbers of benign and darknet network flows containing application layer data are 102,480 and 21,041, respectively. The network flows of darknet traffic in CDarknet2020 can be divided into eight categories, i.e., Audio-Stream, Browsing, Chat, Email, P2P, File-Transfer, Video-Stream, and VOIP. Detailed descriptions of the darknet applications are shown in Table 3.

In the following subsections, we start by introducing the experimental settings used for evaluation purposes. After that, we present a comprehensive evaluation of DarknetSec from the following four perspectives. First, we perform experiments to discuss the influences of several hyperparameters and their corresponding optimal choices. Second, we conduct an ablation study to evaluate the contribution of each component of DarknetSec to the final classification performance. Third, we verify the effectiveness of DarknetSec by comparing it with several state-of-the-art methods in both binary and multiclass classification scenarios. Finally, we demonstrate the generalizability and adaptability of DarknetSec on conventional encrypted traffic classification tasks using two other public datasets. Note that the experiments in Section 4.2, 4.3, and 4.5 involve only multiclass classification scenarios, while

Table 3
Detailed descriptions of the darknet applications in CICDarknet2020.

Application type	Detailed description
Audio-Stream	Vimeo and Youtube
Browsing	Firefox and Chrome
Chat	ICQ, AIM, Skype, Facebook and Hangouts
Email	SMTPS, POP3S and IMAPS
P2P	uTorrent and Transmission (BitTorrent)
File-Transfer	Skype, FTP over SSH (SFTP) and FTP over SSL (FTPS) using Filezilla and an external service
Video-Stream	Vimeo and Youtube
VOIP	Facebook, Skype and Hangouts voice calls

both binary and multiclass classification scenarios are discussed in Section 4.4, which presents detailed performance comparisons with other state-of-the-art methods.

4.1. Experimental settings

4.1.1. Methods for comparative evaluation

To fully demonstrate the effectiveness of DarknetSec in terms of darknet traffic classification, we choose six existing representative methods for comparison, which are briefly introduced as follows. Note that the parameters of all these methods are fine tuned to obtain the best classification performance on the evaluation datasets.

FlowPic, which transforms the basic byte data of each network flow into a grayscale image and uses a typical 2D CNN to classify the encrypted network traffic and identify the application in use. FlowPic provides a generic representation learning model for encrypted traffic classification and application identification, where a LeNet-5-style architecture is leveraged to build the classifier (Shapira and Shavitt, 2021).

CETAnalytics, which extracts high-level feature representations for encrypted traffic classification from both payload content and payload statistics. A Bi-GRU network and a cascaded model composed of a 1D CNN and a Bi-GRU network are used to realize the payload statistics and payload content analytics, respectively. Additionally, to improve accuracy and reduce dimensionality, a simple attention layer is introduced to the output of the feature extraction module to create the final feature vector for classification (Dong et al., 2020).

BGRUA, which uses a Bi-GRU network and an attention mechanism to classify web services running on HTTPS connections. Note that the author only uses three consecutive packets of a network flow and 900 bytes for each packet to build the classifier (Liu et al., 2020b).

VGG19+RF, which transforms the time-based features of darknet traffic into three-dimensional images and uses a pre-trained neural network model to perform feature extraction. After that, a traditional machine learning method is introduced for classification. The combination of VGG19 and RF is chosen as the best combination (Singh et al., 2021).

RF, which addresses traffic classification for anonymous services (Tor, I2P, JonDonym) in both flow-based and packet-based classification scenarios. For the flow-based scenario, 74 flow-based statistics are extracted, and several supervised classification algorithms are employed for classification. Here, we consider only the flow-based classification scenario and chose the RF algorithm for comparison since it achieves the highest accuracy with respect to the classification of anonymous services (Montieri et al., 2018).

DIDarknet, which uses CICFlowMeter to extract 80 statistical features from each network flow of darknet traffic, after which an RF-based feature selection method is employed to select the most important features, which are then converted into grayscale images and fed into a 2D CNN for classification (Habibi Lashkari et al., 2020).

4.1.2. Cross-validation

We use 10-fold cross-validation for the comparison experiments. We randomly divide the dataset into ten disjoint subsets of equal size. For each experiment, we perform training and testing ten times, selecting one subset as the test set and the remaining nine subsets as the training set each time. We use the average value of the ten tests as the final result. Note that all experiments are conducted on a personal computer with an Intel i7-8700K 3.7 GHz CPU, an NVIDIA GeForce GTX1650 4 GHz GPU and 32 GB of memory.

4.1.3. Implementation details

As shown in Section 3.2, the size of $\mathbf{x}_{content}$ is $N * M$, which denotes the first N packets of each network flow and the first M bytes of each packet. Each Conv1D block represents a 1D CNN model. Specifically, for the Conv1D block directly connected to $\mathbf{x}_{content}$, we set the number of convolution kernels to 128, the kernel size to 3, the step size to 1, and the padding field to 'SAME'. Each of the following four parallel Conv1D blocks has 32 convolution kernels, each with the same step size of 1 and the same padding field of 'SAME'. However, the kernel sizes of the four parallel Conv1D blocks are set to 1, 3, 5 and 7, respectively. DarknetSec uses several convolution kernels of different sizes to extract multiple local spatial features for each network flow and combines the outputs of the four Conv1D blocks to obtain fine-grained spatial feature representations.

The input and output size of the self-attention layer after the Conv1D block are both $128 * N$. We set the number of hidden units to 256 for the Bi-LSTM network and set the number of heads to 8 for the multi-head self-attention module. The output sizes of the Bi-LSTM network and the multi-head self-attention module are both $128 * 256$. The attentive content feature fusion module consists of a self-attention layer and a dense layer, and the output sizes of these two layers are $16 * 256$ and 256, respectively.

The statistical feature learning module consists of an MLP with a 64-unit hidden layer and a 32-unit output layer. The MLP in the classification layer has a hidden layer, and the number of hidden units is set to 64.

We employ the Adam optimizer with a learning rate of 0.001 for DarknetSec. Adam is an efficient stochastic optimization algorithm that performs step-by-step optimization on a stochastic objective function with adaptive low-order moment estimation. Additionally, DarknetSec is executed with 30 training epochs and a batch size of 32.

4.1.4. Evaluation metrics

To comprehensively evaluate the classification performance of the proposed architecture, five well-known metrics are adopted for our experiments, i.e., the Accuracy (*Acc*), False Positive Rate (*FPR*), Precision (*Pre*), Recall (*Rec*), and F1-score (*F1*), which are widely used in classification-related tasks. The abovementioned metrics are based on four basic indicators, namely True Positive (*TP*), True Negative (*TN*), False Positive (*FP*) and False Negative (*FN*). Additionally, *Macro-Pre*, *Macro-Rec* and *Macro-F1* are the average

values of precision, recall, and F1-score values for all of each class in the multiclass classification scenario, respectively.

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \quad (27)$$

$$FPR = \frac{FP}{FP + TN} \quad (28)$$

$$Pre = \frac{TP}{TP + FP} \quad (29)$$

$$Rec = \frac{TP}{TP + FN} \quad (30)$$

$$F1 = \frac{2 * Pre * Rec}{Pre + Rec} \quad (31)$$

4.2. Parameter optimization of DarknetSec

The selection of hyperparameters has a significant impact on the performance of DarknetSec. In this subsection, we discuss the selection of several important hyperparameters, including the number of packets N for each network flow and the number of bytes M for each selected packet used to generate the content features, the size of packet length sequence L for the sequential features, and the focusing parameter β of the focal loss.

4.2.1. The numbers of packets (N) and bytes (M)

The number of packets N and the number of bytes M used to generate content features can directly influence the execution efficiency and accuracy of the neural network. Therefore, we conduct experiments to obtain the optimal values of N and M . To reduce the complexity of the training process, we choose smaller values of N and M . We construct various training and test sets with $N = 15, 20, 25, 30, 35, 40, 50, 60, 70, 80, 90, 100$ and $M = 64, 128, 256, 512, 1500$ (consistent with the Maximum Transmission Unit (MTU)). Furthermore, the other hyperparameters are set as $L = 100$, and $\beta = 2$.

We choose the average accuracy of 10-fold cross-validation as the evaluation metric. An exhaustive grid search with a total of 60 experiments is conducted for all values of N and M . The experimental results obtained with different N and M values are presented in Fig. 5. When M reaches 256, the increase in the number of bytes for each selected packet has a very weak effect on the resulting accuracy. Thus, we set M as 256 to ensure high accuracy with a moderate training overhead. As shown in Fig. 5, the accuracies yielded with a varying number of packets selected for each network flow are presented with a red curve when M is set to 256. When only the first 15 packets of each network flow are utilized ($N = 15$), DarknetSec achieves a multiclass accuracy of 86.15%. With the increase in the number of selected packets, the accuracy rate significantly improves when provided with more packets to generate content features. When N exceeds 30, the accuracy is maintained at approximately 92.22% and tends to be stable with no significant change. Therefore, we choose $N = 30$, and $M = 256$ for the following experiments.

4.2.2. The size of packet length sequence L

For the extraction of sequential features, the size of packet length sequence L needs to be specified. To make full use of the sequence characteristics of the packets in a network flow, we consider a larger range of L from 15 to 300 and use the average accuracy of 10-fold cross-validation as the evaluation metric. Various values of L are selected from the set of $\{15, 20, 30, 40, 50, 60, 70, 80, 90, 100, 120, 150, 180, 200, 250, 300\}$.

Meanwhile, we set N , M , and β as 30, 256 and 2, respectively. As shown in Fig. 6, the accuracy gradually improves with increasing L . When L reaches 100, the best accuracy rate emerges and begins to remain stable. Thus, $L = 100$ is selected as the optimal value for the subsequent experiments.

4.2.3. The hyperparameters of the focal loss function

The focal loss function has two adjustable parameters, namely, α and γ . α represents the weight of each class in the training set. It is obvious that compared with the majority classes, the minority classes with fewer training samples are more difficult to classify accurately. Therefore, the minority classes need to be assigned larger weights. In this paper, α is commonly defined as the reciprocal of the category ratio for each class.

In the following discussion, we focus on the influence of γ , which represents how much the classification model pays attention to the minority classes that are difficult to classify. Generally, γ should be set as a natural number. When $\alpha = 1$ and $\gamma = 0$, the focal loss function degenerates into the standard cross-entropy loss function.

We conduct experiments to select the suitable value of γ for the imbalanced dataset of DarknetSec. γ is successively set as 1, 2, 3, 4, and 5, and the other hyperparameters are set as $N = 30$, $M = 256$, and $L = 100$. Additionally, since the focal loss function mainly focuses on alleviating the problem of class imbalance, in addition to the accuracy metric, we also calculate the *Macro-Pre*, *Macro-Rec* and *Macro-F1* of each class. Fig. 7 presents the abovementioned four metrics by calculating the average of the 10-fold cross-validation results obtained with different values of γ . We can see that $\gamma = 2$ achieves the best performance and slightly outperforms the other results when γ is set to larger than 2. We can also conclude a value of γ that is too large may result in overfitting. These experiments demonstrate that the most suitable value of γ is 2.

4.3. Ablation study

DarknetSec contains several components for content and side-channel feature extraction that use hybrid deep learning algorithms. We want to evaluate the contribution of each component to the final classification performance.

As shown in Table 4, we can obtain an *Acc* of 76.56% and a *Macro-F1* of 77.94% by leveraging only the sequential features, while the performance of the statistical features is slightly better than that of the sequential features. By considering both feature sets, namely, by using side-channel features, the *Acc* and *Macro-F1* can be increased to 84.26% and 84.75%, respectively. However, since the payload content of packets is nonexistent in the side-channel features, this approach may not be able to provide enough sufficient information for accurately predicting application types.

A more interesting finding is that we can obtain an *Acc* of 85.42% and a *Macro-F1* of 85.41% by using the content features only; these values are even higher than those yielded by the combination of statistical features and sequential features, which demonstrates that the payload content of packets plays a greater role than payload statistics when applied to encrypted traffic analysis. Furthermore, if we combine the content features and the side-channel features, the *Acc* and *Macro-F1* can be improved to 87.68% and 87.84%, respectively. This also verifies that the fine-grained analysis of encrypted traffic by comprehensively considering both context features and side-channel features can effectively facilitate the accurate classification of darknet traffic.

We next discuss the impact of the self-attention mechanism, including the multi-head self-attention module and the self-attention layer embedded in the local spatial-temporal feature

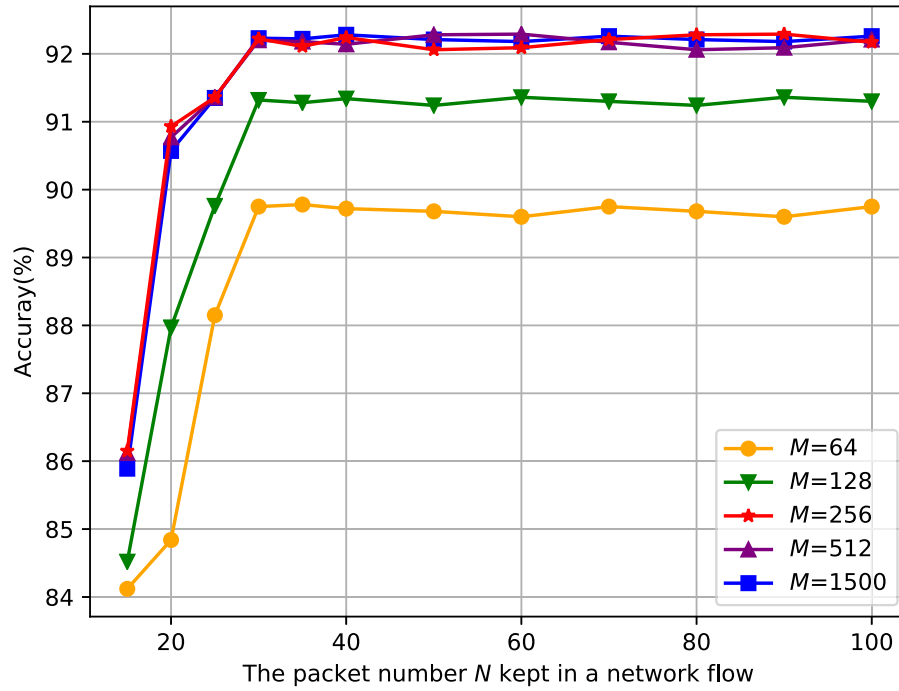


Fig. 5. The average Acc of 10-fold cross-validation with different values of N and M .

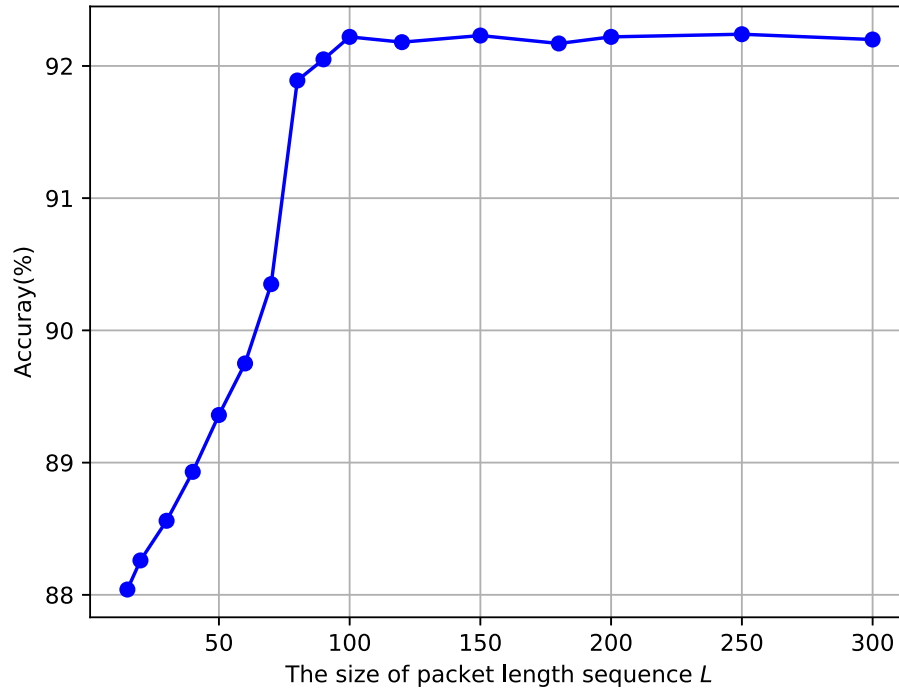


Fig. 6. The average Acc of 10-fold cross-validation with different values of L .

Table 4

Descriptions of the evaluation results regarding the different feature learning components of DarknetSec.

Feature learning components	Acc	Macro – Pre	Macro – Rec	Macro – F1
Statistical features	78.13 ± 2.34	80.54 ± 3.15	77.82 ± 2.56	79.16 ± 2.79
Sequential features	76.56 ± 3.72	78.63 ± 2.80	77.27 ± 1.92	77.94 ± 2.22
Side-channel features (statistical + sequential features)	84.26 ± 2.18	84.64 ± 2.42	84.86 ± 1.94	84.75 ± 2.06
Content features	85.42 ± 2.11	85.46 ± 1.62	85.38 ± 2.04	85.41 ± 1.87
Content + side-channel features	87.68 ± 1.31	87.56 ± 0.82	88.12 ± 1.04	87.84 ± 0.98
Content + side-channel features + multi-head self-attention module	91.28 ± 0.64	91.96 ± 0.70	90.67 ± 1.18	91.31 ± 1.04
All	92.22 ± 0.51	93.36 ± 0.62	90.88 ± 1.24	92.10 ± 0.92

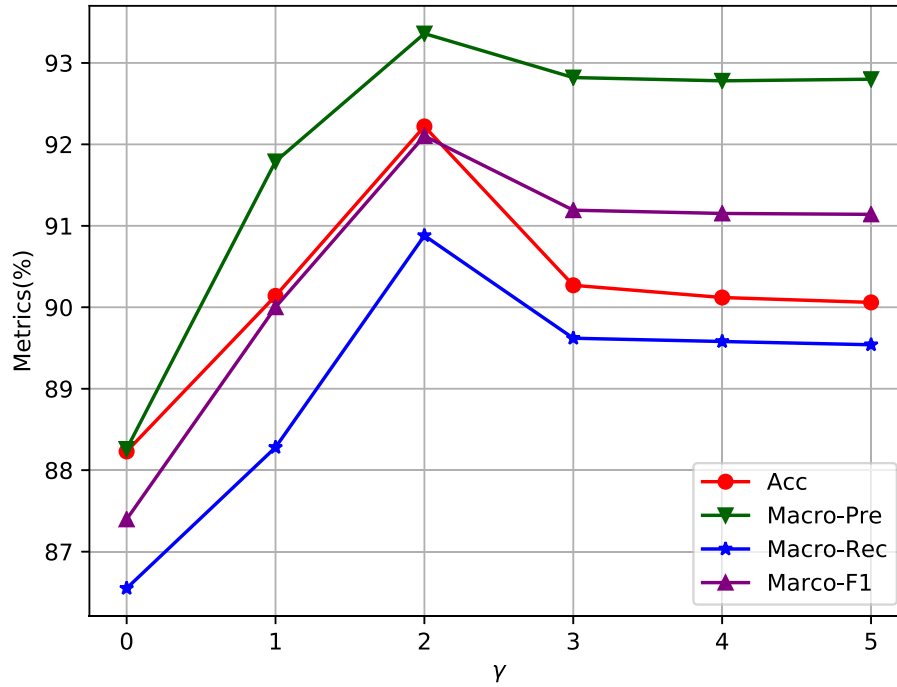


Fig. 7. The average Acc, Macro – Pre, Macro – Rec and Macro – F1 of 10-fold cross-validation with different values of γ .

learning module, on darknet traffic classification. On the basis of using a combination of content features and side-channel features (with an Acc of 87.68% and a Macro – F1 of 87.84%), the application of multi-head self-attention module significantly increases the Acc and Macro – F1 by 3.60% and 3.47%, respectively. The experimental results demonstrate that the multi-head self-attention module can effectively mine the inherent dependencies and hidden connections among the local spatial-temporal features extracted from the input encrypted traffic, thus contributing to providing more discriminative feature representations for fine-grained darknet traffic classification. Finally, by embedding a self-attention layer into the local spatial-temporal feature learning module to help the neural network pay more attention to the most important spatial-temporal feature variables, the Acc and Macro – F1 are increased from 91.28% and 91.31% to 92.22% and 92.10%, respectively.

4.4. Performance comparison with existing methods

In this subsection, we evaluate the performance of DarknetSec relative to several representative methods in binary and multiclass classification scenarios. We apply 10-fold cross-validation and take the average values as the final results.

4.4.1. Binary classification

In terms of the binary classification scenario, all application types of darknet traffic are treated as a single class. This scenario aims to distinguish between benign and darknet network flows. It is worth noting that if a network flow is labeled with the darknet type, we can further employ the multiclass classification model (see Section 4.2.2) to identify the specific application type to which it belongs.

We use the Acc, Pre, Rec and F1 metrics to evaluate the classification results of all these methods. The average metrics obtained via 10-fold cross-validation are presented in Fig. 8. DarknetSec significantly outperforms the other methods in all evaluation metrics. By jointly considering the accuracy and F1-score values obtained on the representative dataset of Darknet2020, we can con-

clude that DarknetSec is superior to its counterparts in terms of the binary classification scenario.

In addition, to further verify the stability of DarknetSec, we randomly select 8000 network flows proportionally from the 8 types of darknet applications (stratified sampling). The ratio of benign and darknet network flows is gradually increased from 1:1 to 9:1. Fig. 9 presents the recall and FPR values of all comparison methods with a varying ratio of benign and darknet network flows. In general, the recall and FPR values of all methods decrease as the proportion of benign network flows increases, and DarknetSec achieves the best classification results when compared with the rest of the methods. Furthermore, with the increase in the proportion of benign network flows, the recall and FPR values of DarknetSec decrease the least. When the ratio of benign and darknet network flows reaches 5:1, the recall and FPR of DarknetSec tend to be stable at approximately 96.20% and 1.44%, respectively. However, the recall values of the remaining methods continue to decrease when the ratio of benign and darknet network flows reaches 9:1, as more darknet network flows are undetected.

The reason for this phenomenon is that DarknetSec extracts more stable and comprehensive feature representations from network flows, which contribute to the accurate identification of darknet flows from a large number of benign flows; in contrast, the other approaches are not sufficiently discriminative to differentiate between benign and darknet network flows. The comparison results show that DarknetSec is superior to the other methods, especially in scenarios with a large amount of background traffic.

4.4.2. Multiclass classification

In the multiclass classification scenario, there are a total of nine classes of network flows, namely, Benign, VoIP, Video-Stream, P2P, Email, File-Transfer, Chat, Browsing and Audio-Stream. We use the Acc, Macro – Pre, Macro – Rec and Macro – F1 metrics to evaluate the classification results of all the tested methods. As shown in Fig. 10, similar to those for the binary classification results, the above four evaluation metrics of DarknetSec are better than those of the other methods. In addition, we provide the accuracy values of each type for all methods, as shown in Table 5. Darknet-

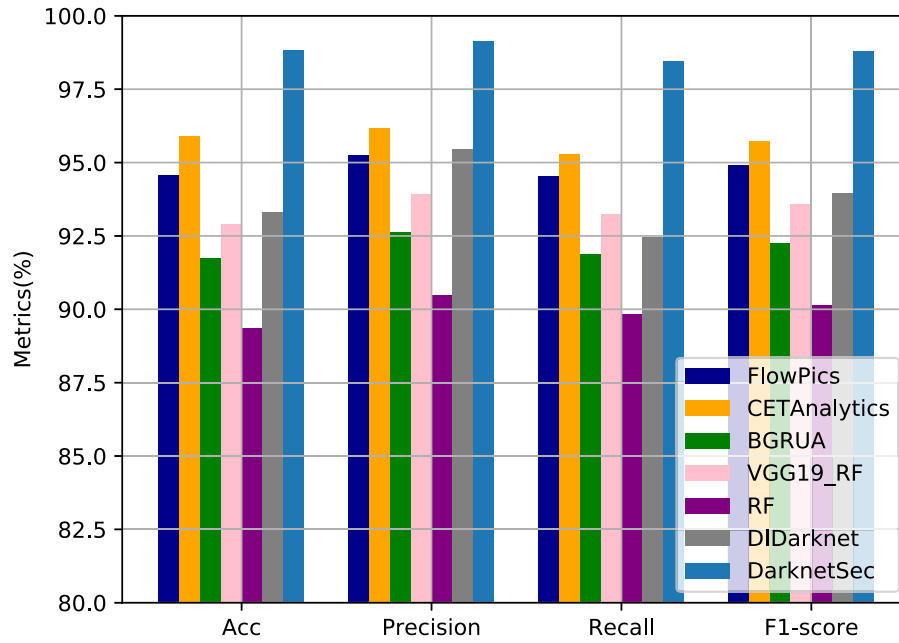


Fig. 8. Binary classification: the average Acc, Pre, Rec and F1 of different methods.

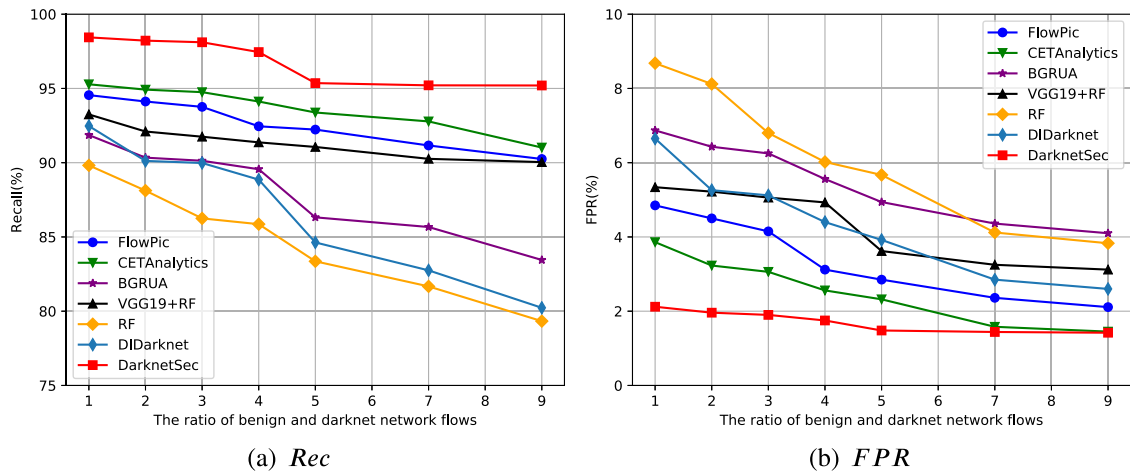


Fig. 9. Binary classification: impact of the ratio of benign and darknet network flows on the Rec and FPR.

Table 5

Multiclass classification: the Acc of different methods for each type of network flow in DarknetSec.

Traffic Type	Methods						
	FlowPic	CETAnalytics	BGRUA	VGG19+RF	RF	DIDarknet	DarknetSec
Benign	98.29 ± 0.73	99.64 ± 0.28	98.22 ± 0.43	99.10 ± 0.36	97.68 ± 1.23	98.66 ± 0.82	99.54 ± 0.54
VoIP	89.58 ± 0.83	90.17 ± 0.49	89.14 ± 0.54	89.92 ± 0.68	89.51 ± 0.39	89.04 ± 0.71	91.35 ± 0.84
Video-Stream	89.22 ± 0.27	89.87 ± 0.91	88.76 ± 0.23	89.58 ± 1.11	86.29 ± 0.66	88.33 ± 0.76	92.56 ± 0.72
P2P	90.64 ± 0.72	92.26 ± 0.54	90.12 ± 0.83	90.34 ± 0.83	88.79 ± 0.63	89.86 ± 0.78	94.69 ± 0.57
Email	79.25 ± 0.93	80.34 ± 0.75	78.33 ± 0.77	79.60 ± 0.71	75.65 ± 1.21	78.13 ± 0.60	81.68 ± 1.05
File-Transfer	90.64 ± 0.72	91.86 ± 0.54	90.12 ± 0.83	91.14 ± 0.83	88.79 ± 0.63	89.86 ± 0.78	94.69 ± 0.57
Chat	94.26 ± 0.35	94.63 ± 0.75	94.86 ± 0.81	93.57 ± 0.64	92.58 ± 0.95	94.23 ± 1.08	95.58 ± 0.63
Browsing	78.05 ± 0.94	78.92 ± 0.85	78.28 ± 0.65	78.34 ± 0.72	77.54 ± 1.21	78.11 ± 1.12	82.74 ± 0.38
Audio-Stream	98.21 ± 0.43	98.32 ± 0.78	97.62 ± 0.35	98.53 ± 0.65	96.24 ± 0.83	97.36 ± 0.16	98.45 ± 0.56

Sec achieves the best classification accuracy values on traffic types except Benign and Audio-Stream. Furthermore, for the Benign and Audio-Stream traffic types, the accuracy values of DarknetSec are only 0.10% and 0.08% lower than the second-place metrics, respectively; this is obviously trivial, and no obvious differences are exhibited. From the fine-grained experimental multiclass classification results, we can obtain several major conclusions as follows.

(1) **DarknetSec outperforms all other approaches, with the highest Acc (92.22%) and Macro – F1 (92.10%) values.** Compared with those of the second-place method, i.e., CETAnalytics, the accuracy and Macro-F1-score values are 2.96% and 3.06% higher, respectively, which shows that our classification architecture has better accuracy and adaptability than other state-of-the-art methods.

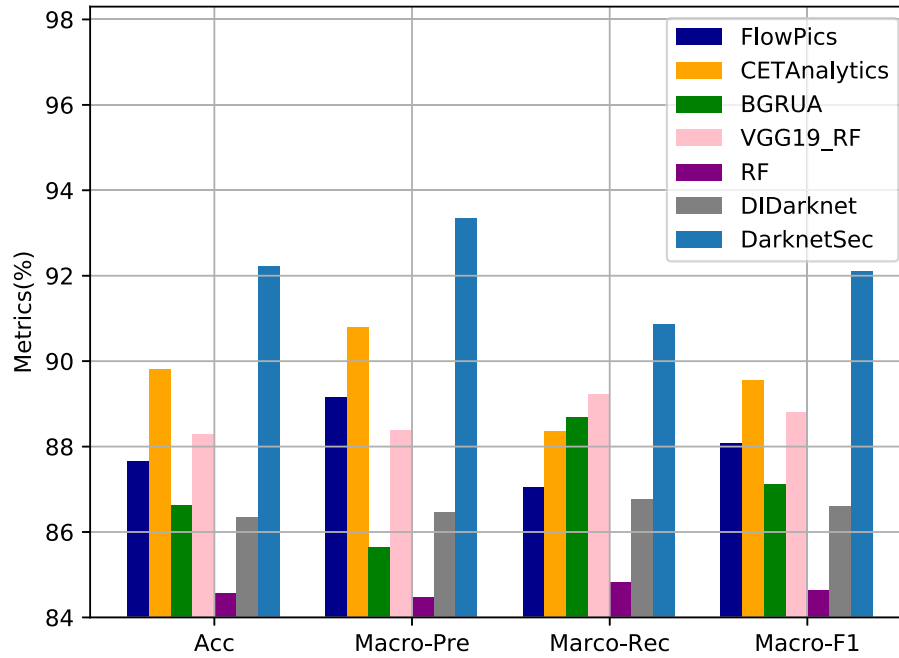


Fig. 10. Multiclass classification: the average Acc, Macro – Pre, Macro – Rec and Macro – F1 of different methods.

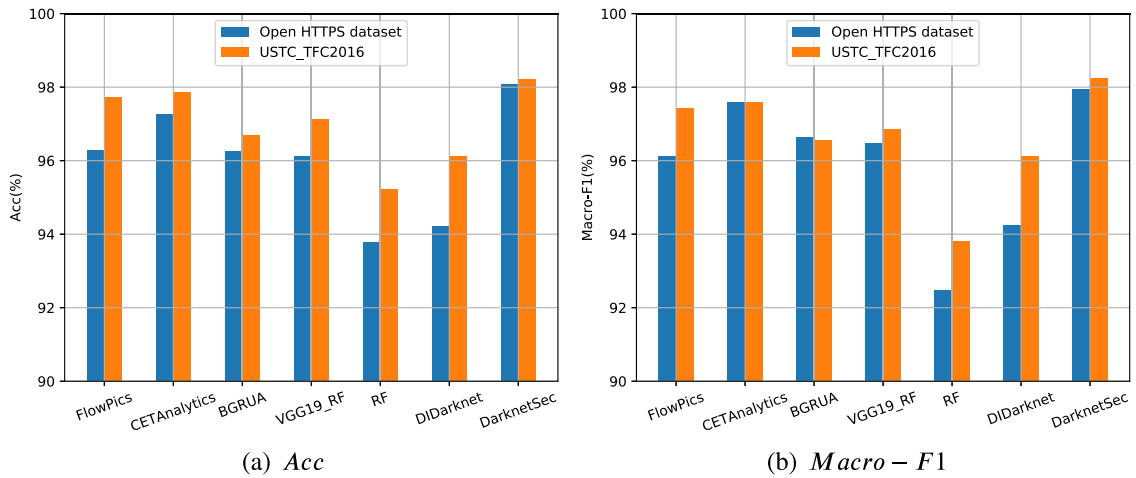


Fig. 11. Multiclass classification results obtained with different methods on USTC-TFC2016 and The Open HTTPS Dataset.

(2) **The multiple deep features, i.e., content features, statistical features, and sequence features, make different contributions to the classification of darknet traffic.** It can be seen that the comprehensive utilization of these three types of features helps DarknetSec achieve the best classification performance.

(3) **The multi-head self-attention module in parallel with the local spatial-temporal feature extraction module helps to capture the global intrinsic relationships among the local spatial-temporal features extracted from different data positions.** Although two other methods, i.e., CETAnalytics and BGRUA, also use the attention mechanism, unlike DarknetSec, they use only a simple attention layer in a cascaded manner after performing feature extraction, which is equivalent to adding a module for the selection of feature importance rather than mining the global inherent dependencies and hidden connections among the extracted local spatial-temporal features.

In summary, the comparison results demonstrate the superiority of DarknetSec in terms of darknet traffic classification and application identification due to the use of the self-attention mechanism and multiple deep feature learning.

4.5. Evaluation on other encrypted traffic classification tasks

Although DarknetSec is proposed to classify the encrypted darknet traffic of VPN and Tor applications, it is still suitable for other encrypted traffic classification tasks. To demonstrate the generalizability of DarknetSec, we conduct comparison experiments on two other public datasets, namely, USTC-TFC2016 (Wang et al., 2017) and The Open HTTPS Dataset (Wazen Shbair, 2016). Specifically, as shown in Table 6, we choose the benign part of the USTC-TFC2016 dataset that has ten traffic types in fourteen pcap files (with a total size of 3.71 GB) and randomly selected ten HTTPS services with more than 1000 network flows from The Open HTTPS Dataset for evaluation. As this is a multiclass classification task, we use the Acc and Macro – F1 metrics to analyze the performance of each method.

Fig. 11 (a) and (b) show the Acc and Macro – F1 values of different methods, respectively. FlowPic, CETAnalytics, BGRUA, VGG19+RF and DarknetSec obtain superior performance, revealing that the feature representations learned by these methods are distinguishable enough to identify the network applications in these

Table 6

Description of USTC-TFC2016 and The Open HTTPS Dataset used to evaluate the generalizability of DarknetSec.

Dataset	Traffic types
USTC-TFC2016 (benign part)	BitTorrent, Facetime, FTP, Gmail, MySQL, Outlook, Skype, SMB, Weibo, WorldOfWarcraft
The Open HTTPS Dataset (10 randomly selected HTTPS services)	www.google.com , r.nexac.com , spanalytics.yahoo.com , ib.adnxs.com , bat.bing.com , www.facebook.com , bat.bing.com , selfrepair.mozilla.org , pixel.quantserve.com , tags.tiqcdn.com

datasets. Among them, DarknetSec slightly outperforms the other methods, demonstrating that DarknetSec is also suitable for the conventional network traffic classification tasks.

5. Conclusion

In this paper, we propose DarknetSec, which can accurately classify the encrypted network flows of darknet traffic using a self-attentive deep learning method. We extract content features from the payload content of packets and side-channel features from payload statistics. Through multiple deep feature learning, we turn the darknet application identification problem into a typical classification task and use hybrid deep learning algorithms to design a powerful classifier. The experiments conducted on the CICDarknet2020 dataset indicate that DarknetSec performs better than other state-of-the-art methods and can distinguish darknet applications from benign applications more accurately. Additionally, we evaluate the effectiveness of DarknetSec on other encrypted traffic classification tasks. In the future work, we plan to use more datasets to verify the effectiveness of DarknetSec and study the impact of network traffic concept drift on the classification performance of DarknetSec.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Jinghong Lan: Conceptualization, Methodology, Validation, Software, Investigation, Data curation, Writing – original draft, Writing – review & editing. **Xudong Liu:** Supervision, Resources, Formal analysis, Writing – review & editing. **Bo Li:** Funding acquisition, Supervision, Project administration. **Yanan Li:** Visualization, Software, Validation. **Tongtong Geng:** Validation, Writing – review & editing.

Acknowledgments

This work was supported by the National Key R&D Program China (2018YFB0803 503), the 2018 joint Research Foundation of Ministry of Education, China Mobile (MCM20180507) and the Opening Project of Shanghai Trusted Industrial Control Platform (TICPSH202003020-ZC).

References

Adewopo, V., Gonen, B., Varlioglu, S., Ozer, M., 2019. Plunge into the underworld: a survey on emergence of darknet. In: 2019 International Conference on Computational Science and Computational Intelligence (CSCI). IEEE, pp. 155–159.

Al-Nabki, M.W., Fidalgo, E., Alegre, E., Fernández-Robles, L., 2019. ToRank: identifying the most influential suspicious domains in the Tor network. *Expert Syst. Appl.* 123, 212–226.

Barratt, M., 2015. A discussion about dark net terminology. *Drugs Internet Soc.* 15.

Biddle, P., England, P., Peinado, M., Willman, B., et al., 2002. The darknet and the future of content distribution. In: *ACM Workshop on Digital Rights Management*, vol. 6, p. 54.

Buxton, J., Bingham, T., 2015. The rise and challenge of dark net drug markets. *Policy Brief* 7, 1–24.

Dong, C., Zhang, C., Lu, Z., Liu, B., Jiang, B., 2020. CETAnalytics: comprehensive effective traffic information analytics for encrypted traffic classification. *Comput. Netw.* 176, 107258.

Fachkha, C., Debbabi, M., 2015. Darknet as a source of cyber intelligence: survey, taxonomy, and characterization. *IEEE Commun. Surv. Tutor.* 18 (2), 1197–1227.

Fang, Y., Li, K., Zheng, R., Liao, S., Wang, Y., 2021. A communication-channel-based method for detecting deeply camouflaged malicious traffic. *Comput. Netw.* 197, 108297.

Habibi Lashkari, A., Kaur, G., Rahali, A., 2020. DiDarknet: a contemporary approach to detect and characterize the darknet traffic using deep image learning. In: 2020 the 10th International Conference on Communication and Network Security, pp. 1–13.

Hassan, M.M., Gumaei, A., Alsanad, A., Alrubaiyan, M., Fortino, G., 2020. A hybrid deep learning model for efficient intrusion detection in big data environment. *Inf. Sci.* 513, 386–396.

Iglesias, F., Zseby, T., 2017. Pattern discovery in internet background radiation. *IEEE Trans. Big Data* 5 (4), 467–480.

Iliadis, L.A., Kaifas, T., 2021. Darknet traffic classification using machine learning techniques. In: 2021 10th International Conference on Modern Circuits and Systems Technologies (MOCAS). IEEE, pp. 1–4.

Kanna, P.R., Santhi, P., 2021. Unified deep learning approach for efficient intrusion detection system using integrated spatial-temporal features. *Knowl. Based Syst.* 226, 107132.

Kumar, S., Vranken, H., van Dijk, J., Hamalainen, T., 2019. Deep in the dark: a novel threat detection system using darknet traffic. In: 2019 IEEE International Conference on Big Data (Big Data). IEEE, pp. 4273–4279.

Lin, K., Xu, X., Gao, H., 2021. TSCRNN: a novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT. *Comput. Netw.* 190, 107974.

Liu, L., Zhen, J., Li, G., Zhan, G., He, Z., Du, B., Lin, L., 2020. Dynamic spatial-temporal representation learning for traffic flow prediction. *IEEE Trans. Intell. Transp. Syst.*

Liu, X., You, J., Wu, Y., Li, T., Li, L., Zhang, Z., Ge, J., 2020. Attention-based bidirectional GRU networks for efficient HTTPS traffic classification. *Inf. Sci.* 541, 297–315.

Montieri, A., Ciunzio, D., Aceto, G., Pescapé, A., 2018. Anonymity services Tor, I2P, JonDonym: classifying in the dark (web). *IEEE Trans. Dependable Secure Comput.* 17 (3), 662–675.

Montieri, A., Ciunzio, D., Bovenzi, G., Persico, V., Pescapé, A., 2019. A dive into the dark web: hierarchical traffic classification of anonymity tools. *IEEE Trans. Netw. Sci. Eng.* 7 (3), 1043–1054.

Niranjana, R., Kumar, V.A., Sheen, S., 2020. Darknet traffic analysis and classification using numerical AGM and mean shift clustering algorithm. *SN Comput. Sci.* 1 (1), 1–10.

Pacheco, F., Exposito, E., Gineste, M., Baudoin, C., Aguilar, J., 2018. Towards the deployment of machine learning solutions in network traffic classification: a systematic survey. *IEEE Commun. Surv. Tutor.* 21 (2), 1988–2014.

Pinheiro, A.J., Bezerra, J.d.M., Burgardt, C.A., Campelo, D.R., 2019. Identifying IoT devices and events based on packet length from encrypted traffic. *Comput. Commun.* 144, 8–17.

Pour, M.S., Mangino, A., Friday, K., Rathbun, M., Bou-Harb, E., Iqbal, F., Samtani, S., Crichigno, J., Ghani, N., 2020. On data-driven curation, learning, and analysis for inferring evolving internet-of-things (IoT) botnets in the wild. *Comput. Secur.* 91, 101707.

Rao, Z., Niu, W., Zhang, X., Li, H., 2018. Tor anonymous traffic identification based on gravitational clustering. *Peer-to-Peer Netw. Appl.* 11 (3), 592–601.

Sarwar, M.B., Hanif, M.K., Talib, R., Younas, M., Sarwar, M.U., 2021. DarkDetect: darknet traffic detection and categorization using modified convolution-long short-term memory. *IEEE Access* 9, 113705–113713.

Shahbar, K., Zincir-Heywood, A.N., 2017. Effects of shared bandwidth on anonymity of the I2P network users. In: 2017 IEEE Security and Privacy Workshops (SPW). IEEE, pp. 235–240.

Shapira, T., Shavitt, Y., 2021. FlowPic: a generic representation for encrypted traffic classification and applications identification. *IEEE Trans. Netw. Serv. Manage.* 18 (2), 1218–1232.

Shen, M., Liu, Y., Zhu, L., Du, X., Hu, J., 2020. Fine-grained webpage fingerprinting using only packet length information of encrypted traffic. *IEEE Trans. Inf. Forensics Secur.* 16, 2046–2059.

Shen, M., Zhang, J., Zhu, L., Xu, K., Du, X., 2021. Accurate decentralized application identification via encrypted traffic analysis using graph neural networks. *IEEE Trans. Inf. Forensics Secur.* 16, 2367–2380.

Singh, D., Shukla, A., Sajwan, M., 2021. Deep transfer learning framework for the identification of malicious activities to combat cyberattack. *Future Gener. Comput. Syst.* 125, 687–697.

Sirinam, P., Mathews, N., Rahman, M.S., Wright, M., 2019. Triplet fingerprinting: more practical and portable website fingerprinting with n-shot learning. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1131–1148.

- Stergiopoulos, G., Talavari, A., Bitsikas, E., Gritzalis, D., 2018. Automatic detection of various malicious traffic using side channel features on TCP packets. In: *European Symposium on Research in Computer Security*. Springer, pp. 346–362.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. In: *Advances in Neural Information Processing Systems*, pp. 5998–6008.
- Wang, W., Zhu, M., Zeng, X., Ye, X., Sheng, Y., 2017. Malware traffic classification using convolutional neural network for representation learning. In: *2017 International Conference on Information Networking (ICOIN)*. IEEE, pp. 712–717.
- Wazen Shbair Thibault Cholez, J. F. I. C., 2016. *Https websites dataset*. <http://betternet.lhs.loria.fr/datasets/https/>.
- Wood, J.A., 2009. The darknet: a digital copyright revolution. *Rich. J. Tech.* 16, 1.
- Xie, G., Li, Q., Jiang, Y., 2021. Self-attentive deep learning method for online traffic classification and its interpretability. *Comput. Netw.* 196, 108267.
- Xie, J., Li, S., Yun, X., Zhang, Y., Chang, P., 2020. HSTF-model: an HTTP-based trojan detection model via the hierarchical spatio-temporal features of traffics. *Comput. Secur.* 96, 101923.
- Yang, J., Zhang, L., Chen, C., Li, Y., Li, R., Wang, G., Jiang, S., Zeng, Z., 2020. A hierarchical deep convolutional neural network and gated recurrent unit framework for structural damage detection. *Inf. Sci.* 540, 117–130.
- Yang, Z.-b., Zhang, J.-p., Zhao, Z.-b., Zhai, Z., Chen, X.-f., 2020. Interpreting network knowledge with attention mechanism for bearing fault diagnosis. *Appl. Soft Comput.* 97, 106829.
- Yao, H., Liu, C., Zhang, P., Wu, S., Jiang, C., Yu, S., 2019. Identification of encrypted traffic through attention mechanism based long short term memory. *IEEE Trans. Big Data.*
- Zheng, H., Lin, F., Feng, X., Chen, Y., 2020. A hybrid deep learning model with attention-based Conv-LSTM networks for short-term traffic flow prediction. *IEEE Trans. Intell. Transp. Syst.*

Jinghong Lan is a PhD candidate in the School of Computer Science and Engineering, Beihang University, China. He received the MS degree in the School of Cyberspace Security from PLA Information Engineering University in Zhengzhou, China. His current research interests include intrusion detection, machine learning, and deception defense.

Xudong Liu is a professor in the School of Computer Science and Engineering, Beihang University, China. His current research interests include machine learning, big data and industrial information security.

Bo Li is an Assistant Professor in the School of Computer Science and Engineering, Beihang University, China. He received the PhD degree in the School of Computer Science and Engineering from Beihang University. His current research interests include industrial information security, mobile and IoT security, and cyber threat intelligence.

Yanan Li is a MSc candidate in the School of Computer and Information Security, Guilin University of Electronic Technology, China. He received the BS degree in the School of Cyberspace Security from PLA Information Engineering University in Zhengzhou, China. His current research interests include intrusion detection, machine learning, and knowledge graph technology.

Tongtong Geng is a PhD candidate in the School of Economics and Management, Xidian University, China. Her received the MS degree in the School of Economics and Management, Xidian University, China. Her current research interests include sharing economy business model, machine learning, and big data research.